SDS/Xerox Sigma 7 — Instruction Register Transfer Language

Notation

Symbol	Meaning
RR[n]	User-visible register n (0–15), mapped to word addresses 0–15
R	4-bit register field register (bits 28–31), loaded from instruction bits 8–11
r	Value held in R; selects destination/source user register
x	X field of instruction (3 bits); index register selector for memory-reference instructions; repurposed as part of immediate operand for immediate instructions
i	I field of instruction (1 bit); 1 = indirect addressing
C	32-bit memory interface register (transparent latch)
D	32-bit secondary ALU input register
A	32-bit primary ALU input register
B	32-bit multiply/divide partner register (forms 64-bit A:B pair)
E	8-bit floating-point exponent register
O	7-bit opcode register
P	19-bit effective address register (bits 15–33)
Q	17-bit next instruction address register (bits 15–31)
CC	4-bit condition code register (bits 1–4)
S	Sum bus — combinational ALU output, no dedicated register
M[addr]	32-bit word memory access at byte address addr
M.H[addr]	16-bit halfword memory access
M.B[addr]	8-bit byte memory access
EA	Effective byte address, held in P after prep phases
sext(v)	Sign extend v to 32 bits

Symbol	Meaning
AWZ	A Was Zero flip-flop; set when low word result is zero during doubleword operations; used in combination with high word result to set CC for full 64-bit zero detection
ENDE	End-of-instruction signal; fires in last execute phase
>> n	Right shift by n bits
<< n	Left shift by n bits
~	Bitwise invert

Common Sequences

ENDE Signal

ENDE fires at the last execute phase of every instruction. It simultaneously fetches the next instruction and sets up registers for the following instruction's prep phases:

```
ENDE: C ← M[Q]                          ; fetch next instruction word
      O ← C[1:7]                         ; load opcode from bits 1-7
      D ← C                              ; instruction word into D via transparent
C latch
      R ← C[8:11]                        ; load R field into 4-bit R register
      P ← P + 4                          ; increment to next instruction byte
address

      ; Set up A and P[32:33] for EA calculation:
      if x=0:              A ← 0;              P[32:33] ← 00
      if x≠0, byte:        A ← RR[x] >> 2;    P[32:33] ← byte_offset
      if x≠0, halfword:    A ← RR[x] >> 1;    P[32:33] ← halfword_offset
      if x≠0, word:        A ← RR[x];         P[32:33] ← 00
      if x≠0, doubleword:  A ← RR[x] << 1;    P[32:33] ← 00
```

Prep Phases

The prep phases calculate the effective address using the instruction word in D, and save the next instruction address to Q:

```
PREP1: Q ← P[15:31]; if i=0 goto PREP3

PREP2: C ← M[C[15:31]]; D ← C           ; indirect: hardware masks C to bits 15-31
                                          ; resolved pointer loaded into D via
transparent C

PREP3: P[15:31] ← S where S ← A + D[15:31]  ; word EA into P[15:31]
                                              ; P[32:33] retains byte offset from
ENDE
```

After PREP3, P holds the complete effective byte address:

- **P[15:31]:** word address from EA calculation
- **P[32:33]:** byte offset within word, set during ENDE

Word Arithmetic Instructions

AW — Add Word (0x30)

```
PREP1-3: EA → P
EX1: A ← RR[r]
EX2: C ← M[P]; D ← C; S ← A + D; A ← S; RR[r] ← S
EX3: CC ← test(A); ENDE
```

SW — Subtract Word (0x38)

```
PREP1-3: EA → P
EX1: A ← RR[r]
EX2: C ← M[P]; D ← C; S ← A + ~D + 1; A ← S; RR[r] ← S
EX3: CC ← test(A); ENDE
```

CW — Compare Word (0x31)

```
PREP1-3: EA → P
EX1: A ← RR[r]
EX2: C ← M[P]; D ← C; S ← A + ~D + 1; A ← S
EX3: CC ← test(A); ENDE
```

AI — Add Immediate (0x20)

```
PREP1: C ← sext(inst[12:31])            ; 20-bit sign-extended immediate (X field
unused)
EX1: A ← RR[r]; D ← C
EX2: S ← A + D; A ← S; RR[r] ← S
EX3: CC ← test(A); ENDE
```

CI — Compare Immediate (0x21)

```
PREP1: C ← sext(inst[12:31])            ; 20-bit sign-extended immediate (X field
unused)
EX1: A ← RR[r]; D ← C
EX2: S ← A + ~D + 1; A ← S
EX3: CC ← test(A); ENDE
```

LI — Load Immediate (0x22)

```
PREP1: C ← sext(inst[12:31])            ; 20-bit sign-extended immediate (X field
unused)
EX1: A ← C
EX2: S ← A; RR[r] ← S
EX3: CC ← test(A); ENDE
```

Halfword Arithmetic Instructions

AH — Add Halfword (0x50)

```
PREP1-3: EA → P                          ; P[32:33] = halfword offset
EX1: A ← RR[r]
EX2: C ← M.H[P]; D ← sext(C[16:31]); S ← A + D; A ← S; RR[r] ← S
EX3: CC ← test(A); ENDE
```

SH — Subtract Halfword (0x58)

```
PREP1-3: EA → P
EX1: A ← RR[r]
EX2: C ← M.H[P]; D ← sext(C[16:31]); S ← A + ~D + 1; A ← S; RR[r] ← S
EX3: CC ← test(A); ENDE
```

CH — Compare Halfword (0x51)

```
PREP1-3: EA → P
EX1: A ← RR[r]
EX2: C ← M.H[P]; D ← sext(C[16:31]); S ← A + ~D + 1; A ← S
EX3: CC ← test(A); ENDE
```

Halfword Load/Store Instructions

LH — Load Halfword (0x52)

```
PREP1-3: EA → P
EX1: C ← M.H[P]; A ← sext(C[16:31])    ; sign extend halfword to 32 bits
EX2: S ← A; RR[r] ← S
EX3: CC ← test(A); ENDE
```

STH — Store Halfword (0x55)

```
PREP1-3: EA → P
EX1: A ← RR[r]
EX2: S ← upward_align_halfword(A)        ; replicate low halfword to both halfword
positions
     M.H[P] ← S                          ; write mask selects correct halfword
```

Note: STH does not update CC.

LCH — Load Complemented Halfword (0x5A)

```
PREP1-3: EA → P
EX1: C ← M.H[P]; D ← sext(C[16:31])
EX2: S ← ~D + 1; A ← S; RR[r] ← S
EX3: CC ← test(A); ENDE
```

LAH — Load Absolute Halfword (0x5B)

```
PREP1-3: EA → P
EX1: C ← M.H[P]; D ← sext(C[16:31])
EX2: if D[0]=0: S ← D;      A ← S; RR[r] ← S; goto EX4
     if D[0]=1: S ← ~D + 1; A ← S; RR[r] ← S
```

```
EX3: RR[r] ← S
EX4: CC ← test(A); ENDE
```

MTH — Modify and Test Halfword (0x53)

```
PREP1-3: EA → P
EX1: C ← M.H[P]; A ← sext(C[16:31])    ; load halfword, sign extend to 32 bits
EX2: D ← sext(R, 16); S ← A + D; A ← S ; add sign-extended 4-bit R field as 16-bit
increment
EX3: M.H[P] ← S                          ; write back modified halfword
EX4: CC ← test(A); ENDE
```

LW — Load Word (0x32)

```
PREP1-3: EA → P
EX1: C ← M[P]; A ← C
EX2: S ← A; RR[r] ← S; P[15:31] ← Q
EX3: P ← P + 4; CC ← test(A); ENDE
```

STW — Store Word (0x35)

```
PREP1-3: EA → P
EX1: A ← RR[r]
EX2: S ← A; M[P] ← S
```

Note: STW does not update CC.

LCW — Load Complemented Word (0x3A)

```
PREP1-3: EA → P
EX1: C ← M[P]; D ← C
EX2: S ← ~D + 1; A ← S; RR[r] ← S
EX3: CC ← test(A); ENDE
```

LAW — Load Absolute Word (0x3B)

```
PREP1-3: EA → P
EX1: C ← M[P]; D ← C
EX2: if D[0]=0: S ← D;       A ← S; RR[r] ← S; goto EX4
     if D[0]=1: S ← ~D + 1;  A ← S; RR[r] ← S
```

```
EX3: RR[r] ← S
EX4: CC ← test(A); ENDE
```

Doubleword Load/Store Instructions

Doubleword operands must be doubleword-aligned: P[32:33] = 00. Register operands use an even/odd pair: RR[r] = high word, RR[r+1] = low word. The **AWZ (A Was Zero)** flip-flop captures whether the low word result was zero, enabling CC to reflect the full 64-bit result:

- **Zero:** AWZ=1 AND high word result is zero
- **Negative:** A[0] of high word result
- **Overflow/Carry:** from high word adder

LD — Load Doubleword (0x12)

```
PREP1-3: EA → P                              ; P[32:33] = 00 (doubleword aligned)
EX1: C ← M[P+4]; A ← C                      ; load low word
EX2: S ← A; RR[r+1] ← S; AWZ ← (S=0)       ; store low word, capture AWZ
EX3: C ← M[P]; A ← C                        ; load high word
EX4: S ← A; RR[r] ← S
EX5: CC ← test(A, AWZ); ENDE                 ; CC reflects full 64-bit result
```

STD — Store Doubleword (0x15)

```
PREP1-3: EA → P
EX1: A ← RR[r]
EX2: S ← A; M[P] ← S                        ; store high word
EX3: A ← RR[r+1]
EX4: S ← A; M[P+4] ← S                      ; store low word
```

Note: STD does not update CC.

AD — Add Doubleword (0x10)

```
PREP1-3: EA → P
EX1: A ← RR[r+1]                             ; low word of register pair
EX2: C ← M[P+4]; D ← C
     S ← A + D; A ← S; RR[r+1] ← S; AWZ ← (S=0)  ; add low words, capture AWZ
EX3: A ← RR[r]                               ; high word of register pair
EX4: C ← M[P]; D ← C
```

```
     S ← A + D + carry; A ← S; RR[r] ← S   ; add high words with carry
EX5: CC ← test(A, AWZ); ENDE
```

SD — Subtract Doubleword (0x18)

```
PREP1-3: EA → P
EX1: A ← RR[r+1]
EX2: C ← M[P+4]; D ← C
     S ← A + ~D + 1; A ← S; RR[r+1] ← S; AWZ ← (S=0)  ; subtract low words
EX3: A ← RR[r]
EX4: C ← M[P]; D ← C
     S ← A + ~D + borrow; A ← S; RR[r] ← S  ; subtract high words with borrow
EX5: CC ← test(A, AWZ); ENDE
```

CD — Compare Doubleword (0x11)

```
PREP1-3: EA → P
EX1: A ← RR[r+1]
EX2: C ← M[P+4]; D ← C
     S ← A + ~D + 1; A ← S; AWZ ← (S=0)    ; compare low words
EX3: A ← RR[r]
EX4: C ← M[P]; D ← C
     S ← A + ~D + borrow; A ← S             ; compare high words with borrow
EX5: CC ← test(A, AWZ); ENDE                 ; result not written back
```

LCD — Load Complemented Doubleword (0x1A)

```
PREP1-3: EA → P
EX1: C ← M[P+4]; D ← C
     S ← ~D + 1; A ← S; RR[r+1] ← S; AWZ ← (S=0)  ; complement low word
EX2: C ← M[P]; D ← C
     S ← ~D + carry; A ← S; RR[r] ← S      ; complement high word with carry
EX3: CC ← test(A, AWZ); ENDE
```

LAD — Load Absolute Doubleword (0x1B)

```
PREP1-3: EA → P
EX1: C ← M[P]; D ← C                       ; load high word to check sign
EX2: if D[0]=0: goto EX5                    ; positive: load both words directly
EX3: C ← M[P+4]; D ← C
     S ← ~D + 1; A ← S; RR[r+1] ← S; AWZ ← (S=0)  ; negate low word
```

```
EX4: C ← M[P]; D ← C
     S ← ~D + carry; A ← S; RR[r] ← S      ; negate high word with carry
     goto EX7
EX5: C ← M[P+4]; A ← C; RR[r+1] ← S; AWZ ← (S=0)  ; load low word, positive case
EX6: C ← M[P]; A ← C; RR[r] ← S            ; load high word, positive case
EX7: CC ← test(A, AWZ); ENDE
```

AND (0x4B)

```
PREP1-3: EA → P
EX1: A ← RR[r]
EX2: C ← M[P]; D ← C; S ← A AND D; A ← S; RR[r] ← S
EX3: CC ← test(A); ENDE
```

OR (0x49)

```
PREP1-3: EA → P
EX1: A ← RR[r]
EX2: C ← M[P]; D ← C; S ← A OR D; A ← S; RR[r] ← S
EX3: CC ← test(A); ENDE
```

EOR — Exclusive OR (0x48)

```
PREP1-3: EA → P
EX1: A ← RR[r]
EX2: C ← M[P]; D ← C; S ← A XOR D; A ← S; RR[r] ← S
EX3: CC ← test(A); ENDE
```

LB — Load Byte (0x72)

```
PREP1-3: EA → P
EX1: C ← M.B[P]; A ← zero_extend(C[24:31])  ; zero extend byte to 32 bits, upper
24 bits = 0
EX2: S ← A; RR[r] ← S
EX3: CC ← test(A); ENDE
```

STB — Store Byte (0x75)

```
PREP1-3: EA → P
EX1: A ← RR[r]
EX2: S ← upward_align_byte(A)                ; replicate low byte to all four byte
```

```
positions
     M.B[P] ← S                                ; write mask selects correct byte
```

Note: STB does not update CC.

CB — Compare Byte (0x71)

```
PREP1-3: EA → P
EX1: A ← zero_extend(RR[r][24:31])           ; low byte of register, zero extended
EX2: C ← M.B[P]; D ← zero_extend(C[24:31]); S ← A + ~D + 1; A ← S
EX3: CC ← test(A); ENDE
```

MTB — Modify and Test Byte (0x73)

```
PREP1-3: EA → P
EX1: C ← M.B[P]; A ← zero_extend(C[24:31])  ; load byte, zero extend to 32 bits
EX2: D ← sext(R, 8); S ← A + D; A ← S       ; add sign-extended 4-bit R field as
8-bit increment
EX3: M.B[P] ← S                               ; write back modified byte
EX4: CC ← test(A[24:31]); ENDE               ; CC set from byte result only
```